TCTools-Build_Web — How to Use It

A beginner's manual for people who have never written code

Version **v0.1.0-rc1** · Windows · macOS · Linux

About the pictures in this manual. This manual shows screenshot placeholders instead of finished screenshots. The real images are published **per release**, so they always match a shipped build rather than a moving checkout, and they live under:

> `docs_Documentation_文档/images_Images_图片/`

Every placeholder below names the file it will become. When your copy of the documentation includes those images, they appear in the same positions, at the same relative paths. Nothing else in the text changes.

A word before you start. You do not need to know what HTML, CSS or JavaScript are. You will meet those words later in this manual, and by the end they will feel ordinary. Everything here is written for someone opening this application for the first time.

---

1. What this is, and what you can make with it

TCTools-Build_Web — short name **TC-BuildWeb** — is a desktop application for building web pages without writing code.

You work in three simple moves:

1. **Drag elements onto a page.** A heading, a picture, a button, a whole navigation bar — pick it from a panel on the left and drop it on the page.
2. **Snap blocks together to add behaviour.** Want the button to do something? Connect coloured blocks that read like sentences: *when this button is clicked, show a message*.
3. **Export plain files you own.** The result is ordinary `.html`, `.css` and `.js`. No account, no subscription, no special format at the destination. You can put the files on any web host, email them, or keep them on a USB stick.

What you can make

You want to build	You can do it here
A personal page or CV	Yes — headings, text, images, links
A small business site	Yes — navbar, hero section, pricing table, footer
A photo gallery	Yes — the ready-made gallery section
A contact form	Yes — a form section, plus an optional starter server to receive it
A page that fetches live data	Yes — HTTP blocks call an API and put the answer on the page
A multi-page site	Yes — add as many pages as you like
A page built entirely by an AI assistant	Yes — TI-UseAgent, with your approval at every change

What it is not

It is not a photo editor, not a spreadsheet, and not a website *host*. It builds the files; putting them online is a separate, easy step described in section 11.

---

2. Installing and starting the app

What you need

Item	Requirement
Operating system	Windows 10 (1709 or later) or Windows 11; macOS on Apple silicon or Intel; Debian or Ubuntu Linux on x86_64 or aarch64
Disk space	A few hundred megabytes once the application and its libraries are installed
Graphics	Any ordinary desktop, laptop or virtual machine with a working display
Internet	Only needed for the AI assistant and for pages that call an API. Design, blocks, preview and export all work offline

Installing

Windows. Run the installer (`.exe`) and follow it. If you were given a portable folder instead, there is nothing to install: open the folder and double-click `TCTools-Build_Web.exe`. Keep the whole folder together — the application needs the DLLs beside it.

macOS. Open the `.dmg`, then drag **TCTools-Build_Web** into your **Applications** folder. The first time you open it, macOS may ask you to confirm: right-click the app and choose **Open**, then confirm once. After that a normal double-click works.

Linux. Install the `.deb` with your software installer, or run the AppImage:

```
chmod +x TCTools-Build_Web-*.AppImage
./TCTools-Build_Web-*.AppImage
```

If you are building from source instead, the full instructions are in the repository's README.md.

Starting it for the first time

Start the application the way you start any program. You will see a welcome screen with three ways in:

- **New project** — start from an empty page.
- **Open project** — open a `.tcbw` file you already have.
- **Recent projects** — reopen something you worked on before.

Welcome screen / 欢迎界面

`../images_Images_图片/13-welcome-screen.png`

Screenshot placeholder · 截图占位符

If a previous session ended unexpectedly, the welcome screen also offers to **recover unsaved work**. The application autosaves as you go and keeps a crash-recovery record, so choosing *recover* brings back the last state it captured. Choosing *discard* throws that record away and leaves your saved files untouched.

Checking that everything is healthy: `--doctor`

If something seems wrong — a missing feature, an odd dialog, a machine you do not trust — run the built-in diagnostic. Open a terminal (PowerShell on Windows, Terminal on macOS, your shell on Linux) and run:

```
# Windows
.\TCTools-Build_Web.exe --doctor
```

```
# macOS and Linux
./TCTools-Build_Web --doctor
```

It prints a short report and exits. Here is the shape of that report, taken from the reference build machine — your versions, paths and counts will differ:

```
TCTools-Build_Web 0.1.0-rc1 (rc1)
Qt runtime      : 6.11.1
Qt build        : 6.11.1
Platform        : windows / Windows 11
Architecture    : x86_64
Kernel          : winnt 10.0.26100
Locale          : en_US
Settings file   : C:\Users\you\AppData\Roaming\turtlelnc\TCTools-Build_Web.ini
Secret backend  : Windows DPAPI (OS-backed)
Vendors         : <number of AI vendor presets in this build>
Models          : <number of known model identifiers>
HTML elements   : <number of entries in the Elements panel>
Block types     : <number of block definitions in the toolbox>
WebEngine       : not available (external browser preview)
```

Read it like this:

- **Secret backend** tells you how your AI credentials are protected. *OS-backed* is the strong case. *obfuscated* means the file is scrambled but not truly encrypted — that is what Linux currently reports, and the application says so honestly rather than pretending otherwise.
- **WebEngine** tells you whether the built-in browser view exists. On a default build it does not, so the preview works as described in section 4 and section 11.
- **Vendors**, **Models**, **HTML elements** and **Block types** are counts. If a count is zero, that part of the product did not load, and reinstalling is the right next step.

If you are reporting a problem to someone, paste this report in. It answers most of the questions they would otherwise have to ask.

---

3. Your first page in 10 minutes

Let us build a real page, from empty window to finished file. Follow along; you cannot break anything.

Step 1 — Make a new project

Press **Ctrl+N** (on macOS, **Cmd+N**), or click **New project** on the welcome screen. A page appears in the middle of the window. Give the project a name when the application asks, or leave it and rename it later with **Ctrl+Shift+S**.

You are now in **Design** mode. The centre of the window is your canvas: a blank page with a dashed drop area that says *Drag an element here to begin*.

Step 2 — Drag in a heading

Look at the left panel. It is called **Elements**, and it groups everything you can put on a page into categories: Layout, Text, Media, Form, Data, Navigation, Advanced — plus a **Ready-made sections** group with finished pieces such as a navbar, a hero, a footer, a pricing table, a gallery, a contact form and a login form.

Open **Text**, find **Heading 1**, and drag it onto the dashed area. Release.

A large heading appears on the canvas. It is already selected — you can tell because it has a selection outline and the right-hand panel is showing its settings.

Design mode / 设计模式

`../images_Images_图片/01-design-mode.png`

Screenshot placeholder · 截图占位符

Step 3 — Edit the text

With the heading selected, look at the **Inspector** on the right. It has three tabs: **Content**, **Style** and **Attributes**.

- **Content** is where the words live.
- **Style** is how they look.
- **Attributes** is lower-level detail you will rarely need.

You are on **Content**. Type your own heading — for example **Owen's Bakery** — into the text box. The canvas updates as you type. There is no save button; your work is recorded automatically, and **Ctrl+Z** undoes anything.

Inspector tabs / 属性面板标签

`../images_Images_图片/03-inspector-tabs.png`

Screenshot placeholder · 截图占位符

Step 4 — Change its colour

Switch to the **Style** tab. You will see:

- **Quick styles** — a row of one-click presets. Try one; if you do not like it, press **Ctrl+Z**.
- **Typography** — font, size, weight, alignment, line height.
- **Colour** — the text colour, and **Background** — the colour behind it.
- **Spacing, corner radius and shadow** — how much room the element gets, how round its corners are, and whether it casts a shadow.

Click the colour field next to **Colour** and pick something. Watch the heading change immediately.

Step 5 — Add a button

Go back to **Elements**, open **Form**, and drag **Button** onto the canvas, just below the heading. Select it, go to **Content**, and change its label to **Say hello**.

If the button landed in the wrong place, drag it on the canvas to move it. Hold **Shift** while dragging to keep the movement straight.

Step 6 — Make the button do something, with blocks

Now the interesting part. Press **Ctrl+2** to switch to **Blocks** mode.

Blocks mode has a toolbox on the left and a script area in the middle. Blocks are colour-coded by family: Events, Interface, Text, Control, Logic, Math, Variables, Storage, HTTP, Backend, Data, AI, MCP Tools, Timing, Console. The last few families are *advanced* and hidden until you turn on **Show advanced blocks**.

Every script begins with a **hat block** — a block with a rounded top, which means *when this happens*. Drag **when button clicked** from the **Events** family into the script area.

Then open **Interface** and drag **show message** underneath it. Blocks snap together when the notch lines up: hover the new block just below the hat block until it clicks into place.

Your script now reads:

```
when button clicked
    show message  "Hello!"
```

Click inside the message block and type your own text. That is a program. It is also, quite literally, what will be compiled into JavaScript for you.

Blocks editor / 积木编辑器

`../images_Images_图片/04-blocks-editor.png`

Screenshot placeholder · 截图占位符

The right-hand panel already shows the **Generated JavaScript** for what you just built, together with **Errors** and **Warnings** badges. If a badge is red, a block is incomplete — usually a blank field.

Step 7 — Preview it in your browser

Press **Ctrl+6** for **Preview** mode. You get two things:

- A **device-framed structural preview** — a quick, approximate look at your page at Desktop, Tablet and Mobile width.
- A **real browser preview** — the application starts a small server on your own computer (at an address like `http://127.0.0.1:8000/`) and opens or serves the page to your ordinary browser, with live reload as you edit.

For anything that matters, use the browser preview. It is the real thing; the in-app frame is an approximation. This manual is honest about that on purpose, and so is the application.

Browser preview / 浏览器预览

`../images_Images_图片/12-preview-browser.png`

Screenshot placeholder · 截图占位符

Step 8 — Export

Press **Ctrl+E**. The export dialog opens and asks what shape you want the output in. For a first page, choose **Single .html file** — one self-contained file you can email or upload anywhere.

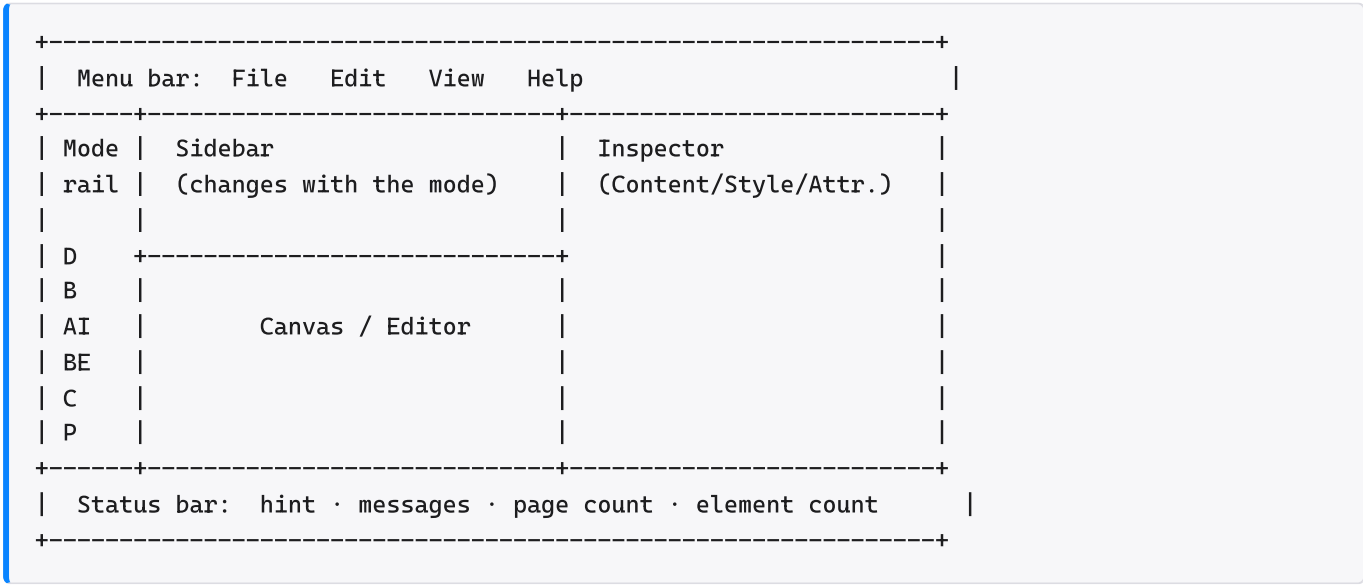
Choose a folder, confirm, and you are done. Your page exists as a file on your own disk, in a format every browser in the world can open.

That was the whole loop. Everything else in this manual is detail on top of the eight steps you just did.

---

4. The window at a glance

The window has five regions. Once you can name them, everything else is easy.



The mode rail

The narrow strip on the far left switches between the six things the application can do. Each entry has a name and a one-line description in the status bar when you hover it.

Mode	Shortcut	What it is for
Design	**Ctrl+1**	Drag elements onto the canvas and style them
Blocks	**Ctrl+2**	Snap blocks together to add behaviour
AI	**Ctrl+3**	Ask TI-UseAgent to build or change something
Backend	**Ctrl+4**	Connect the page to a server or an API
Code	**Ctrl+5**	Read and export the generated source
Preview	**Ctrl+6**	See the page the way a browser will

The mode rail / 模式侧栏

`../images_Images_图片/16-mode-rail.png`

Screenshot placeholder · 截图占位符

The sidebar

The sidebar changes with the mode, because each mode needs different things at hand:

Mode	Sidebar shows
Design	**Pages** and **Layers** — your page list, and the tree of everything on the current page
Blocks	**Scripts** — every script in the project, one stack per entry
AI	**Pages** — what the assistant can look at
Backend	**Endpoints** — the requests your page can make
Code	the generated **Files**
Preview	the preview controls

You can hide the sidebar and the inspector from the **View** menu when you want a bigger canvas.

The canvas

The canvas is the working surface. In Design mode it is your page; in Blocks mode it is the script area; in Preview mode it is the device frame.

Design mode also has a small toolbar for **zoom** (25 % to 200 %, plus a reset button), a **grid** overlay you can toggle, and the **breakpoint** selector — **Desktop**, **Tablet**, **Mobile**.

Breakpoints / 断点

`../images_Images_图片/15-breakpoints.png`

Screenshot placeholder · 截图占位符

The inspector

The inspector is the right-hand panel. In Design mode it shows **Content**, **Style** and **Attributes** for whatever is selected. In other modes it shows that mode's output: the generated JavaScript in Blocks mode, the provider configuration in AI mode, the response in Backend mode, the file list in Code mode.

Elements panel / 元素面板

`../images_Images_图片/02-elements-panel.png`

Screenshot placeholder · 截图占位符

The status bar

The bottom strip tells you what just happened and where you are: a hint or a transient message, the number of pages, and the number of elements in the current page. When something succeeds or fails, the message appears here first.

---

5. Designing

The Elements panel

Elements are grouped into categories:

Category	Typical contents
Layout	containers, sections, columns, dividers, spacers
Text	headings, paragraphs, lists, quotes, code, labels
Media	images, figures, video, audio, icons
Form	forms, inputs, text areas, selects, checkboxes, buttons
Data	tables, definition lists, progress bars
Navigation	nav bars, link lists, breadcrumbs, pagination
Advanced	iframes, canvases, custom containers, raw passthrough
Ready-made sections	navbar, hero, footer, pricing, gallery, contact form, login form

A ready-made section is a whole group of elements — you drop it once and then edit the pieces individually. It is the fastest way to get a page that looks finished.

Dragging, selecting, moving

- **Drag** an element from the panel onto the canvas. A drop indicator shows where it will land.
- **Click** an element to select it.
- **Drag a selected element** on the canvas to move it. Hold **Shift** to keep the movement straight.
- **Snapping** helps you line things up. The canvas toolbar shows whether snapping is on.

Selecting a thin or nested element can be fiddly. Use **Select parent** from the canvas context menu to step outward one level at a time.

Layers and nesting

The **Layers** section of the sidebar shows the page as a tree. A section contains a container, which contains a heading and a button — that is the shape of nearly every page. The tree lets you select something that is hard to click, and it shows which elements can contain others.

Two useful moves:

- **Wrap** — put the selected element inside a new container, so you can style the group as a whole.
- **Reorder** — move the selected element earlier or later among its siblings.

Undo, copy, paste and duplicate

Undo is unlimited within a session and works across every editor: whether you changed something by dragging, by typing in the inspector, or by asking the AI, the same **Ctrl+Z** takes it back. **Ctrl+Shift+Z** or **Ctrl+Y** redoes.

Cut, **Copy** and **Paste** live in the **Edit** menu. This release deliberately does not bind them to Ctrl+X / Ctrl+C / Ctrl+V: those keys keep their normal meaning inside every text box on the page, and stealing them would be worse than saving you a menu click. **Ctrl+D** duplicates the selected element in place.

Locking and hiding

Every element has two flags:

- **Locked** — the element stays visible but can no longer be moved or edited by accident. Use it once a piece is right.
- **Hidden** — the element is not shown on the canvas and is not exported.

Both are toggled from the inspector, and both are ordinary undoable actions.

Breakpoints: Desktop, Tablet and Mobile

A breakpoint is a screen width at which the design is allowed to change. The three in this application are:

Breakpoint	Think of it as
Desktop	a laptop or a large monitor
Tablet	a tablet held upright
Mobile	a phone

Switch with the breakpoint selector, or press **Ctrl+0** to jump straight back to Desktop.

Desktop is the base. Whatever you set while on Desktop applies everywhere unless you override it.

A mobile override is a deliberate exception. Suppose your heading is 48 px on Desktop. You switch to Mobile, select the heading, and set 28 px. Now:

- Desktop: 48 px
- Tablet: **48 px** — inherited, because you never touched Tablet
- Mobile: 28 px — your override

Switch back to Desktop and change the size to 40 px, and Desktop and Tablet follow to 40 px while Mobile stays at 28 px. That is the whole idea: *one base value, plus explicit exceptions where you want them.* To remove an override, clear the field while that breakpoint is selected.

6. Styling

Select anything and the inspector's three tabs do the rest.

Content

The words, the image address, the button label, the link target — whatever the element is *about*. This is where beginners spend most of their time, and it is meant to be.

Style

Group	What it controls
Quick styles	one-click presets: a heading style, a card look, a button look
Typography	font family, size, weight, alignment, line height, letter spacing
Colour	text colour, background colour, transparency
Spacing	the space around the element and inside it
Corner radius	how rounded the corners are — 0 is a sharp box, larger is softer
Shadow	a soft shadow behind the element, which makes cards float
Border	width, style, colour
Layout	how children are arranged: flowing one after another, or placed freely

Attributes

Attributes are the technical details behind an element: its `id`, its `class` names, its `href`, its `alt` text, and any custom data you want to attach. Two of them are worth knowing as a beginner:

- `alt` on an image is the text a screen reader reads aloud. Write something meaningful; it also appears if the image fails to load.
- `id` is the element's unique name. The AI and the blocks refer to elements by `id`, so renaming one is a real change.

CSS units without the jargon

Every size in styling ends with a small unit. There are only five you need.

Unit	Plain meaning	When to use it
`px`	picture elements — one dot on the screen	borders, fine detail, anything you want exactly this big
`%`	a share of the parent's size	widths: `100%` means "as wide as my container"
`em`	a multiple of **this** element's font size	spacing that should grow with the text
`rem`	a multiple of the **page's** base font size	consistent spacing everywhere
`vw` / `vh`	a share of the **viewport** width / height	full-screen banners: `100vh` is one screen tall

Two habits that save trouble:

- Prefer `rem` for spacing and `%` for widths. They adapt when the screen changes.
- `100vh` on a phone is slightly taller than what you can see, because of the browser's address bar. If a full-height section feels wrong on mobile, try `90vh` and check it in the browser preview.

---

7. Programming with blocks

What a hat block is

A **hat block** is the block with a rounded top and no notch above it. It means *when this happens*. Every script starts with exactly one hat block:

- **when page loaded** — runs once, as soon as the page is ready
- **when button clicked** — runs when a particular button is pressed
- **when input changed** — runs when a form field changes
- **every N seconds** — runs on a timer

Without a hat block a stack of blocks is not a script, and it will not run.

How blocks snap

Blocks have a notch on top and a matching bump on the bottom. Drag a block near another one and it snaps into place when the notches line up. Some blocks also have **slots** — oval spaces on the right — into which you drop a value block such as a number, a piece of text, or a variable.

If a block refuses to snap, it is usually because the slot expects a value and you are holding a statement. Colour is a good clue: values and statements are drawn differently.

The block families

Family	What it is for
Events	the hat blocks — what starts a script
Interface	showing and hiding things, changing text, reading values
Text	joining text, taking it apart, changing case
Control	if, if/else, repeat, wait, stop
Logic	comparisons and and/or/not
Math	arithmetic, rounding, random numbers
Variables	named boxes that remember a value
Storage	remembering values in the browser between visits
HTTP	calling an API and reading the answer
Backend	endpoints you defined in Backend mode
Data	lists and records: adding, reading, counting
AI	asking the configured model for a short answer
MCP Tools	tools from MCP servers you have trusted
Timing	delays and repeating timers
Console	printing messages, which is how you see what your script is doing

Show advanced blocks is off by default. Turn it on when you want the later families; beginners rarely need them on day one.

Variables

A variable is a named box. *Set* a variable to put something in the box; *change* it to add to what is there; use the variable block to read it out. Names are yours to choose, so `score`, `visitorName` and `totalPrice` are all fine.

Variables live while the page is open. If you want a value to survive a reload, use the **Storage** family instead.

If and loops

```
if  score > 10
    show message  "You win!"
else
    show message  "Keep going"
```

```
repeat 5
    change  counter  by 1
```

These read almost exactly like English, which is the point. Nest them by dropping one inside the other.

Calling an API with the HTTP blocks

An **API** is a web address that answers with data instead of a page. The pattern is always the same three steps:

1. **Request** — the HTTP block that says *get* or *post* to a URL.
2. **Wait for the answer** — the block that hands you the response when it arrives.
3. **Use it** — read a value out of the answer and put it on the page.

If you defined the address in **Backend** mode, the **Backend** family gives you a ready-made block for it, so you do not have to retype the URL.

Reacting to a click

The classic first program:

```
when button clicked
    set  greeting  to  "Hello, " + name field value
    set text of  #output  to  greeting
```

Read it as a sentence: *when the button is clicked, put a greeting into the* `greeting` *variable, then make the element whose id is* `output` *show it.* That is all a "click handler" is.

The compiled-code panel

Blocks mode shows the JavaScript your blocks became, in the **Generated JavaScript** panel. This is not a hidden translation step — the panel is showing you the actual output that the Code mode and the export will use.

Above it are two badges:

- **Errors** — something is wrong and the code cannot be produced. Usually an empty field in a block. Fix it and the error clears immediately.
- **Warnings** — the code was produced, but something looks off. Read them; they often catch a mistake before a visitor does.

Compiled JavaScript panel / 编译后的 JavaScript 面板

`../images_Images_图片/05-compiled-code.png`

Screenshot placeholder · 截图占位符

You never have to read the JavaScript. It is there so that you *can*.

---

8. Connecting a backend

The words, first

- An **endpoint** is one address your page can talk to, such as `https://example.com/api/contact`. It has a method (usually `GET` to read or `POST` to send) and, for `POST`, a body — the data you send.
- A **backend** is the program that answers at that address.
- A **REST** endpoint is the ordinary kind: a URL plus a method. A **GraphQL** endpoint is a single URL where you send a query describing what you want.

You do not need a backend to publish a page. You need one as soon as a form has to send something anywhere, or a page has to read data that changes.

Defining an endpoint

Press **Ctrl+4** for **Backend** mode and click **Add endpoint**. Fill in:

Field	What to put
Name	a short label for you, for example `contact`
Method	`GET`, `POST`, `PUT`, `PATCH` or `DELETE`
URL	the full address, including `https://`
Content type	`application/json` for almost everything modern
Headers / Query	extra request headers and URL parameters, if the API wants them
Body	for `POST`, the JSON you send — you can use placeholders for form values
Authorisation	how the request proves who it is: a header, a bearer token, a query parameter
Timeout	how long to wait before giving up

The application validates as you type and tells you what is missing, so you are never left guessing whether the address is well formed.

Backend endpoints / 后端端点

`../images_Images_图片/09-backend-endpoints.png`

Screenshot placeholder · 截图占位符

Sending a test request

Click **Send test request**. The response panel shows the **status** code, the **time** it took, the **body** and the **headers**.

Status	Meaning in plain words
200	fine, here is your data
201	created — your `POST` worked
400	the server did not understand what you sent
401 / 403	the server does not accept your credentials
404	no such address
500	the server itself broke

Testing inside the builder matters: it separates *my page is wrong* from *the server is wrong* before you have written a single block.

If you are entering real credentials, note that they are stored outside the project file, so sharing the `.tcbw` file never shares your key.

Generating a matching starter server

Click **Create a matching server**. The application offers a list of server platforms and writes a small, runnable project that already exposes the endpoints you defined — so the page you just built works end to end.

The catalogue covers **20 targets**, from frameworks you run yourself to hosted platforms:

Family	Targets
Node.js	Node.js + Express, Node.js + Fastify, static JSON server, Directus, Netlify Functions, Vercel Functions
Python	Python + FastAPI, Python + Flask
Go, Rust, PHP, Java, .NET	Go net/http, Rust + Axum, PHP + Laravel, Java + Spring Boot, .NET Minimal API
Hosted platforms	Supabase, Firebase Functions, Cloudflare Workers, Deno Deploy, PocketBase, Appwrite, Nhost (Hasura)

Seven of them have a **dedicated generator** that writes a complete starter project: **Node.js + Express**, **Python + FastAPI**, **Cloudflare Workers**, **Supabase (Edge)**, **Firebase Functions**, **Go net/http** and the **static JSON server**. For any other target the application still produces a working dependency-free static JSON server, and says so in a warning rather than failing silently — an honest fallback rather than a broken promise.

Starter server targets / 起始服务器目标

`../images_Images_图片/10-scaffold-targets.png`

Screenshot placeholder · 截图占位符

A complete worked example: a contact form posting to Node/Express

1. Build the form. In Design mode, drag the **Contact form** ready-made section onto the canvas. It brings a name field, an email field, a message box and a submit button, already labelled and styled.

2. Define the endpoint. Switch to Backend (**Ctrl+4**) and add:

Field	Value
Name	`contact`
Method	`POST`
URL	`http://127.0.0.1:3000/api/contact`
Content type	`application/json`
Body	`{"name": "", "email": "", "message": ""}`

Test it now and you will get a connection error — that is correct, because the server does not exist yet. Skipping this step is the most common reason people think their form is broken.

3. Generate the server. Click **Create a matching server**, choose **Node.js + Express**, and write it to a folder next to your project. Open that folder and run:

```
npm install
npm start
```

The server listens on `http://127.0.0.1:3000` and already has a route for `/api/contact`, plus a `/health` address you can open in a browser to confirm it is alive.

4. Test again. Back in Backend mode, send the test request a second time. Now you get a real response — a status code and a JSON body. You have proved the whole chain works before touching a single block.

5. Wire the button. Go to Blocks mode and give the submit button a script:

```
when submit button clicked
    set  result  to  post to contact with the form values
    if  result status = 200
        show message  "Thank you — we will be in touch."
    else
        show message  "Something went wrong. Please try again."
```

6. Preview and breathe. Preview in your browser (**Ctrl+6**), fill the form in, press submit, and watch the thank-you message appear while the server logs the request it just received.

7. Export. Export the page (**Ctrl+E**) and keep the generated server folder beside it. When you put the page online, point the endpoint URL at wherever the server ends up running — the application reminds you, because a `127.0.0.1` address only ever means *this computer*.

---

9. Using the AI (TI-UseAgent)

TI-UseAgent is the assistant built into the application. It is genuinely useful — and it is designed so that you stay in charge.

Choosing a provider

Press **Ctrl+3** for **AI** mode. The right-hand panel configures the provider:

- **Region filter** — **International**, **China** or **Local / self-hosted**.
- **Vendor search** — type a few letters to narrow a long list.
- **Model picker** — choose which model to talk to.
- **Sign-in method** — API key, OAuth, or a device code.
- **Approval policy** — how much the assistant may do without asking.
- **Available tools** — the exact list of actions it may take.
- **MCP servers** — extra tools from outside programs (see section 10).

There are presets for **52+ vendors**: international ones such as OpenAI, Anthropic, Google Gemini, Azure OpenAI, AWS Bedrock, Mistral, Cohere, xAI, Groq, Together, Fireworks, Replicate, Hugging Face, DeepSeek, OpenRouter and Cerebras; Chinese providers such as Qwen (DashScope), Zhipu GLM, Moonshot Kimi, Baidu Qianfan, Tencent Hunyuan, ByteDance Doubao, MiniMax, 01.AI, Baichuan, StepFun, iFlytek Spark, SenseTime, SiliconFlow, ModelScope and Huawei ModelArts; and local servers such as Ollama, LM Studio, vLLM, llama.cpp, Jan and text-generation-webui.

A preset is a *starting point*, not a promise. Vendors change their addresses and model names all the time, so every field stays editable — if a preset has drifted, correct it rather than working around it.



Signing in

Three ways, in increasing order of sophistication:

API key. The simplest. You create a key on the vendor's website, paste it into the application, and press **Save key**. Presets link you to the right page of the vendor's console. The key is stored **outside your project file**, so you can share a `.tcbw` file without sharing your key.

OAuth sign-in. You press **Sign in with OAuth**, a browser window opens, you approve the request on the vendor's own site, and the application receives a token. Under the hood this is OAuth 2.0 with PKCE — the modern, safer variant, where no client secret has to be embedded in the application.

Device code. For devices where a browser is awkward. The application shows a short code; you enter it on the vendor's verification page; the application picks up the token when you are done.

Bearer tokens are supported too. Whichever route you use, the assistant itself **never receives your raw credential** — it only ever gets to ask for an action.

The approval prompt, and what Allow means

Here is the part worth reading twice.

Every action the assistant can take is a **declared tool** with a description and a strictly typed input. It cannot do anything that is not on that list, and it cannot invent a new one.

Tools that change your project pass through an **approval gate**. When one is about to run, you see a prompt naming the tool, what it will do, and where:

> **TI-UseAgent wants to make a change** Review it before it happens. Nothing is changed until you allow it.
>
> **Allow · Don't allow**

- **Allow** — the change happens. It goes through exactly the same editing path as a change you made by hand, which means **Ctrl+Z undoes it** just like anything else.
- **Don't allow** — nothing happens. The assistant is told you declined and can propose something else.

Allow and **Don't allow** apply to that one action. There is also an allowing-everything option in the prompt for a run of related changes, and a deny-everything option when you have seen enough.

The default policy is **Allow additions, ask before deletions**: adding elements and pages happens without interrupting you, while anything that removes work asks first. The other three policies are:

Policy	Meaning
Never let the AI change anything	read-only; it may look, but not touch
Allow additions, never deletions	additive changes run automatically; deletions are refused outright
Ask before every change	nothing happens without your click
Allow additions, ask before deletions	the default

Read-only tools — listing pages, reading an outline, finding an element — never prompt. Being asked permission to *look* would train you to click through prompts without reading them.

Approval prompt / 审批提示

`../images_Images_图片/07-approval-prompt.png`

Screenshot placeholder · 截图占位符

The tool list

The panel lists every action the assistant may take, grouped by area, and marks each one as **read-only**, **changes your page**, or **deletes content**. Read it once when you first sign in. It is the honest answer to "what can this thing actually do?"

Prompts that work well

Be concrete, and say what you want rather than what you do not want:

Instead of	Try
"make it nicer"	"make the heading dark blue, 40 px, and centre it on mobile"
"add a form"	"add a contact form under the hero, with name, email and message"
"fix the button"	"the Say hello button does nothing when clicked — make it show a message"
"something's wrong"	"the pricing section looks cramped on mobile; give it more space"

The assistant can read the page structure, so you can point at things: *"the second card in the pricing section"*. It works best on one clear change at a time, and it will tell you when a request is ambiguous rather than guessing.

What the AI will refuse to do

The assistant is deliberately limited, and these limits are features:

- **It cannot delete things without asking** under the default policy — and under the two stricter deletion policies, it cannot delete at all.
- **It cannot see your API key**, your OAuth token, or any other credential. It asks for actions, not secrets.
- **It cannot do anything not on the tool list.** No arbitrary code, no shell commands, no file system wandering.
- **It cannot reach outside your project** except where a declared tool says so.
- **It cannot promise that its output is correct.** Model output is untrusted input. You own what you publish, and the review step is yours, not optional.
- **It will not help you attack something.** It is a page builder, not a hacking tool.

If a refusal seems wrong, read the prompt again — nine times out of ten the tool it declined is not the one you thought you were asking for.

---

10. MCP servers

What MCP is, in plain language

MCP stands for *Model Context Protocol*. It is a common way for an AI assistant to use tools that live in other programs.

An ordinary tool is built into this application — *add an element*, *list pages*. An MCP server is a separate program that offers its own tools: a database reader, a design-system lookup, a company's internal search. MCP is the plug shape they all share, so the assistant can use any of them without special code for each one.

Two kinds are supported:

- **stdio** — a program on your own computer, which the application starts and talks to directly.
- **HTTP** — a server somewhere, reached over the network.

The trust prompt, and why it exists

A **stdio** MCP server is a program that will run on your computer with your account's permissions. That is a serious thing to agree to.

So the application refuses to start an MCP server until you have explicitly marked it **trusted**. Until then the panel says **Not trusted — connection blocked**, and the Connect button does nothing. There is no "trust everything" switch, because that would defeat the purpose.

Trust means: *I have looked at what this program is, and I am willing to run it.* Only trust servers you installed yourself or that come from a source you actually know. Anything you would not double-click, do not trust.

MCP servers panel / MCP 服务器面板

`../images_Images_图片/08-mcp-panel.png`

Screenshot placeholder · 截图占位符

Adding a stdio server

1. Open the AI panel's **MCP servers** section and click **Add MCP server**.
2. Give it a short **server id** — letters and digits, for example `notes`.
3. Choose the **stdio** transport and enter the command that starts the program, with any arguments and environment variables it needs.
4. Save. The entry now shows as **not trusted**, and is connected to nothing.
5. Read what you just entered. If you recognise and want to run it, tick **Trust this server**.
6. Click **Connect**. The panel reports how many tools the server offers.

Secrets for an MCP server are passed through its **environment**, never on the command line — command lines show up in process lists, environment blocks do not.

Namespaced tool names

An MCP tool is never called by the name the server gave it. It is renamed:

```
mcp_<server>_<tool>
```

So a `search` tool from a server with the id `notes` becomes `mcp_notes_search`.

This is not cosmetic. Without it, a server could offer a tool called `delete_page` and quietly shadow a built-in tool of the same name — and you would approve what you thought was the harmless built-in. The namespace makes every MCP tool unmistakably an outsider, both in the tool list and in the approval prompt.

MCP tools pass through the same approval gate as everything else. A trusted server gives its tools the *right to be offered*, not the right to act.

11. Exporting and publishing

The four export formats

Press **Ctrl+E**, or use **Code** mode (**Ctrl+5**) and click **Export**.

Format	What you get	Use it when
Single .html file	one self-contained file: markup, styles and script inside it	you want to email it, upload one file, or keep a copy that always travels complete
HTML + CSS + JS	`index.html` plus separate stylesheet and script files	you or someone else will edit the code later; this is the tidiest shape for a developer
JavaScript (.js) only	a script that builds the page in the browser at runtime	you are embedding the page in something else, or feeding it to another tool
Complete static site	everything, plus assets, a manifest and a readme	you are publishing a real site and want no surprises

What ends up in the folder

With the complete static site you get roughly this:

```
my-site/
├── index.html            the home page
├── about.html            one file per page you made
├── styles.css            the stylesheet
├── app.js                the script
├── manifest.json         what was generated, and when
├── README.txt            how to publish it
└── assets/               images and other files
```

The exported markup is ordinary HTML, the styling is ordinary CSS and the behaviour is ordinary JavaScript. There is no runtime, no lock-in format, and no account required to serve them. The files are yours.

Export dialog / 导出对话框

`../images_Images_图片/11-export-dialog.png`

Screenshot placeholder · 截图占位符

Putting it online

Whichever host you choose, the step is the same: put the exported folder's contents somewhere public, make sure one page is named `index.html`, and point people at the address.

Route	How	Good for
GitHub Pages	create a repository, upload the exported files, then enable Pages in the repository settings	free hosting you already understand
Netlify drop	drag the exported folder onto their drop page in a browser	the fastest possible result, no account gymnastics
Any static host	upload the folder over FTP, SFTP or a web control panel	a host you already pay for
Your own computer, for testing	use the preview server, or any local web server	checking on another device on the same network

Two things people trip over:

- `index.html` **is the front door.** If your home page exported under another name, either rename it or set the host's default document to that name.
- **A** `127.0.0.1` **address only works on your own machine.** If your page calls an endpoint at `http://127.0.0.1:3000/...`, visitors' browsers will look for a server on *their* computer. Update the endpoint to the real address of your deployed server before you publish.

After that, your page is on the internet, built with files you own.

---

12. Keyboard shortcuts

Every shortcut below was checked against the application's own source. On macOS, use **Cmd** wherever you see **Ctrl**.

Shortcut	What it does
Ctrl+N	New project
Ctrl+O	Open a project
Ctrl+S	Save
Ctrl+Shift+S	Save as
Ctrl+E	Export
Ctrl+Z	Undo
Ctrl+Shift+Z or **Ctrl+Y**	Redo
Ctrl+D	Duplicate the selected element
Ctrl+K	Ask the AI — opens AI mode and puts the cursor in the message box
Ctrl+1	Design mode
Ctrl+2	Blocks mode
Ctrl+3	AI mode
Ctrl+4	Backend mode
Ctrl+5	Code mode
Ctrl+6	Preview mode
Ctrl+,	Settings
Ctrl+0	Switch to the Desktop breakpoint
Esc	Deselect

Three notes that save confusion:

- **Delete and Backspace are not global shortcuts.** Removing an element is handled by the canvas, which only listens when no text box has focus. This is deliberate: a global Delete would steal the key from every field you type in. To delete, select the element and use **Delete** in the Edit menu, or the delete control on the canvas.
- **Cut, Copy and Paste** are in the **Edit** menu and are not bound to Ctrl+X / Ctrl+C / Ctrl+V, so those keys keep working normally inside text.
- **Ctrl+0** resets the breakpoint, not the zoom. Zoom is set from the canvas toolbar.

---

13. Troubleshooting

Symptom	Likely cause	What to do
Nothing happens when I drag an element	The pointer is over a panel that is not a drop target, or you are dragging from a category heading rather than an element row	Drop onto the canvas itself, where the dashed *Drag an element here to begin* area is. Category headings open and close a group; the rows underneath are the draggable items
The preview is blank	No page is selected, or the current page has no elements yet, or you are looking at the in-app frame while a breakpoint hides everything	Check the page list in the sidebar and pick a page. Add one element so there is something to show. Switch breakpoints to Desktop (**Ctrl+0**). If it is still blank, use the browser preview, which is authoritative
The preview server will not start	The port is already in use — often by a previous run of the application that is still open, or by another program	Stop the preview and start it again; the application picks the next free port. Close other copies of the application. If it still refuses, run `--doctor` and check the report before reporting a bug
My API key was rejected	The key is wrong, expired, revoked, for a different vendor, or the vendor expects a different sign-in method	Re-copy the key from the vendor's console with no leading or trailing space. Confirm the provider preset matches the vendor that issued it. Try the vendor's own console or a plain HTTP client with the same key to prove where the fault is. Remember keys expire and get rotated
The model list is empty	No provider is selected, the vendor's model endpoint needs credentials the application does not have yet, or the preset has drifted from the vendor's current API	Choose a provider first, then sign in — some vendors only list models to an authenticated caller. Type a model name by hand if the preset's list is stale, and check the vendor's documentation for the current identifiers
The AI keeps asking for approval	The approval policy is set to ask before every change, or your request genuinely involves deletions	Open the AI panel and change the approval policy. If you want additions to proceed silently and only deletions to ask, choose **Allow additions, ask before deletions** — that is the default and the calmest setting
Exports look unstyled	The stylesheet did not travel with the markup — common when a multi-file export is copied partly, or when a single file was opened from inside an archive	Use **Single .html file** if you need one portable file. Otherwise copy the **whole** exported folder, keeping `styles.css` and `app.js` beside `index.html`. Open the exported page in a browser and check the developer console for a missing-file error
An MCP server will not connect	The server is not trusted, or the command or address is wrong, or the program is not installed	Confirm the entry says **Trust this server** and that you have ticked it — an untrusted server is blocked by design. Check the command runs on its own in a terminal. For an HTTP server, check the address and whether it needs a token. Read the panel's error message; it names the step that failed

Symptom	Likely cause	What to do
The whole window is blank or the app closes at once	The user interface failed to load — usually a bad installation	Reinstall, and run `--doctor` to capture the report
`--headless --export` **fails**	`--headless` was given without `--export`	Always pass both: `--headless --export <folder>`. `--headless` alone is an error by design and exits with status 3
Unsaved work disappeared after a crash	The recovery record was discarded, or the project was never saved once	On the next start, choose **recover** on the welcome screen. Then save properly with **Ctrl+S** so there is a real file next time
Text shows as empty boxes	A font is missing for that language	Install a font that covers the language (Microsoft YaHei or Noto Sans CJK for Chinese) and restart the application

--doctor output / --doctor 输出

`../images_Images_图片/14-troubleshooting-doctor.png`

Screenshot placeholder · 截图占位符

14. Glossary

Term	What it means
HTML	*HyperText Markup Language* — the language that says what is on a page: a heading, a paragraph, an image. It describes structure, not looks
CSS	*Cascading Style Sheets* — the language that says how things look: colour, size, spacing, roundness. It describes appearance
JavaScript	the language that makes a page *do* things: react to a click, fetch data, show a message. It describes behaviour
Element	one piece of a page — a heading, an image, a button. In this application, something you drag from the Elements panel
Attribute	extra information attached to an element, such as an image's `alt` text or a link's `href` address. Written as name and value
Block	a colour-coded piece of a program in Blocks mode. Snaps to other blocks and reads like a sentence
Script	one stack of blocks, starting with a hat block. One script is one small piece of behaviour
Endpoint	one web address your page can talk to, with a method such as GET or POST
API	*Application Programming Interface* — a service that answers requests with data rather than with a web page. An endpoint is one of its doors
API key	a long secret string that identifies you to a service. Treat it like a password: never paste it into a page you publish
OAuth	a way to sign in to one service using another, without ever handing your password to the application. It is how "Sign in with…" buttons work
MCP	*Model Context Protocol* — a common plug shape that lets an AI assistant use tools provided by separate programs
Breakpoint	a screen width at which the design is allowed to change. This application has Desktop, Tablet and Mobile
Responsive	a page that adapts to the screen it is on, instead of assuming one size. Breakpoints are how you make one responsive

---

15. Getting help and reporting a problem

Before you ask

Three minutes of preparation turns a frustrating report into a fix:

1. **Run** `--doctor` and copy the whole report. It answers most environment questions in one paste.

2. **Write down the exact steps** that led to the problem — what you clicked, in order. "It broke" is hard to act on; "I set the breakpoint to Mobile, changed the font size, and the canvas went blank" can be reproduced.
3. **Try the browser preview.** If the page is fine there, the problem is in the in-app frame; if it is broken there too, the problem is in the page.
4. **Say what you expected** as well as what happened.

Where to ask

- **Bugs and feature requests** — open an issue on the project's GitHub page, turtlelnc/TCTools-BuildWeb. Include the `--doctor` report and your steps.
- **Security-sensitive reports** — do not open a public issue. Follow the security guidance in the repository's README.md and use private vulnerability reporting.
- **In the application** — the **Help** menu opens the About dialog, which shows the version, the attribution and the current keyboard shortcut list, so you can check the shortcuts against the build you are actually running.

Two honest notes about this release

This is **v0.1.0-rc1**, a release candidate. Being straight with you:

- **The in-app preview is structural, not a browser.** A default build has no embedded browser engine, so the in-app frame is an approximation. Your own browser, through the preview server, is the authoritative view — that is why the application gives you both.
- **On Linux, the credential store is obfuscated, not encrypted.** The key sits in a file beside the data. It stops casual inspection and nothing more. On Windows it is sealed with DPAPI and bound to your account; on macOS the values live in your login Keychain. The application reports which one you have, and never claims more than that.

Generated with DeepSeek V4.1 Flash on DeepSeek Harness.

本项目由 DeepSeek V4.1 Flash 在 DeepSeek Harness 上生成。

TCTools-Build_Web v0.1.0-rc1 · MIT License · Documentation images ship per release under `docs_Documentation_文档/images_Images_图片/`.